

Known Issues

All known release issues related to this project will be documented in this file.

If you have a known issue that is not listed here, please open an [issue on the project's GitHub repository](#).

Table of Contents

- **1.0 Control Center never becomes Ready when it wins the race against Kafka's DNS**
 - **1.1 Symptom**
 - **1.2 Cause**
 - **1.3 Workaround**
 - **2.0 Every image pull fails with TLS handshake timeout (ErrImagePull / ImagePullBackOff)**
 - **2.1 Symptom**
 - **2.2 Cause**
 - **2.3 Workaround**
-

1.0 Control Center never becomes Ready when it wins the race against Kafka's DNS

Affects: `cp-enterprise-control-center-next-gen:2.5.0` on minikube (`make cp-core-up`)

1.1 Symptom

`make c3-open` reports a healthy port-forward, but the browser shows `ERR_CONNECTION_REFUSED` on `http://localhost:9021`. `kubectl get pods -n confluent` shows the pod stuck at **2/3 Running with 0 restarts** — it never crash-loops and never recovers on its own:

NAME	READY	STATUS	RESTARTS	AGE
controlcenter-0	2/3	Running	0	45m

The readiness probe fails continuously:

```
Readiness probe failed: Get "http://10.244.0.7:9021/2.0/status/app_info":  
connect: connection refused
```

1.2 Cause

All Confluent Platform pods are applied at once, so Control Center can start before the `kafka` headless Service has endpoints. C3 resolves `bootstrap.servers` once, eagerly, with no retry — when DNS is not ready yet, its `main` thread dies during Guice provisioning:

```
WARN [main] Couldn't resolve server kafka:9071 from bootstrap.servers as
DNS resolution failed for kafka
Exception in thread "main" com.google.inject.ProvisionException:
  Caused by: KafkaException: Failed to create new KafkaAdminClient
  Caused by: ConfigException: No resolvable bootstrap urls given in
bootstrap.servers
```

Non-daemon Kafka Streams threads keep the JVM alive after `main` exits, so the container never terminates, the kubelet never restarts it, and it never binds port `9021`. The pod stays in this state indefinitely.

Two details make this hard to spot:

- The **fatal exception is in the middle of the log, not at the tail** — `kubectl logs --tail` shows only `Unable to obtain lock as state directory is already locked by another process`, which is a downstream symptom of the dead `main` thread, not the cause. Grep for `No resolvable bootstrap urls` to confirm.
- `kubectl port-forward` attaches to any *Running* pod regardless of *readiness*, so the port-forward itself genuinely succeeds while nothing is listening on the other end.

1.3 Workaround

Restart the pod once Kafka is up; the StatefulSet recreates it and it reaches 3/3 in about a minute:

```
kubectl delete pod controlcenter-0 -n confluent
kubectl wait --for=condition=Ready pod/controlcenter-0 -n confluent --
timeout=240s
make c3-open
```

`make c3-open` is gated on `make c3-ready`, which waits for the pod to report Ready (up to `C3_READY_TIMEOUT`, default `180s`) and prints this diagnosis instead of opening a browser onto a dead port.

2.0 Every image pull fails with `TLS handshake timeout (ErrImagePull / ImagePullBackOff)`

Affects: minikube on the **legacy Docker driver** under Docker Desktop for Mac, when Docker Desktop is routing egress through its built-in proxy (typically while a VPN is connected)

The Makefile now runs minikube as a VM (`vfkit` on macOS, `kvm2/qemu` on Linux) with containerd, and it never goes through Docker Desktop, so this issue only affects clusters created before that change. The permanent fix is to move to the VM driver: `make minikube-delete && make minikube-start`. The same symptom can also appear on the VM driver: the Mac itself reaches Docker Hub, but traffic forwarded from the VM gets no reply data back. All sites fail from inside the node, while ping and the TCP connect still succeed. The likely cause is a third-party network extension or content filter on the host (check System Settings → Network → Filters & Proxies, and

`systemextensionsctl list`). If you can't remove it, run any HTTP proxy that supports **CONNECT** on the Mac (for example `tinyproxy` or `squid`), listening on an address the VM can reach and allowing its subnet. For example, with `tinyproxy`:

```
brew install tinyproxy
# In $(brew --prefix)/etc/tinyproxy/tinyproxy.conf: set 'Port 3128' and add
'Allow 192.168.64.0/24'
```

Then set `MINIKUBE_HTTP_PROXY` for this machine in a git-ignored `local.mk` (start from `local.mk.example`):

```
MINIKUBE_HTTP_PROXY ?= http://192.168.64.1:3128 # the Mac as seen from
the vfkite VM
```

From then on, every target uses the proxy. `make minikube-start` starts the `tinyproxy` Homebrew service, passes `--docker-env`, and points `containerd` + `BuildKit` in the node at the proxy. `make minikube-stop` and `make minikube-delete` stop the service again (set `MINIKUBE_PROXY_BREW_SERVICE` empty to manage it yourself). For a node that's already running, use `make minikube-proxy-apply`. To go direct for a single run, use `MINIKUBE_HTTP_PROXY= make ...`.

The proxy then opens the outbound connections as a host process, which works. The option also covers `make flink-image-build` (base-image pulls and the `Containerfile`'s `curl` fetch stage). `MINIKUBE_NO_PROXY` keeps cluster-internal traffic off the proxy.

2.1 Symptom

The first pod to start — usually `confluent-operator` during `make cp-up` — never pulls its image, and `make cp-watch` cycles between `ErrImagePull` and `ImagePullBackOff`:

NAME	READY	STATUS	RESTARTS
AGE confluent-operator-6dbff485d6-kwq6n 2m37s	0/1	ImagePullBackOff	0

`kubectl describe pod` shows the pull timing out against Docker Hub itself, not a missing tag:

```
Failed to pull image "docker.io/confluentinc/confluent-operator:0.1718.99":
Error response from daemon:
  Get "https://registry-1.docker.io/v2/": net/http: TLS handshake timeout
```

Yet `docker pull` on the Mac works fine. Every other image in the stack (Kafka, Schema Registry, Control Center, `cp-flink`, `RustFS`, `aws-cli`) fails the same way.

2.2 Cause

Docker Desktop is configured with an internal HTTP proxy (`docker info` shows **HTTPS Proxy: `http.docker.internal:3128`**). Only the **Docker Desktop daemon** uses it — that is why host-side pulls succeed. Containers, including the minikube node container and the Docker daemon running *inside* it, egress directly, and with a VPN tunnel (`utun*`) up on the host that direct path stalls every TLS handshake. DNS resolves and TCP 443 connects, so it looks like a registry outage rather than a local routing problem.

Quick confirmation — direct egress from the node times out, the same request through the proxy returns **401** (Docker Hub's normal unauthenticated answer):

```
minikube ssh -- "curl -s -o /dev/null -w '%{http_code}\n' --max-time 10
https://registry-1.docker.io/v2/"
# 000
minikube ssh -- "curl -s -o /dev/null -w '%{http_code}\n' --max-time 10 -x
http://http.docker.internal:3128 https://registry-1.docker.io/v2/"
# 401
```

2.3 Workaround

Point the Docker daemon inside minikube at the same proxy, keeping cluster-internal traffic off it, and restart that daemon. Pods in `ImagePullBackOff` recover on their next retry:

```
minikube ssh -- "sudo mkdir -p /etc/systemd/system/docker.service.d &&
printf
'[Service]\nEnvironment=\"HTTP_PROXY=http://http.docker.internal:3128\"\nEn
vironment=\"HTTPS_PROXY=http://http.docker.internal:3128\"\nEnvironment=\"N
O_PROXY=localhost,127.0.0.1,10.96.0.0/12,10.244.0.0/16,192.168.49.0/24,.svc
,.cluster.local,hubproxy.docker.internal\"\n' | sudo tee
/etc/systemd/system/docker.service.d/http-proxy.conf && sudo systemctl
daemon-reload && sudo systemctl restart docker"
```

The drop-in survives `minikube stop / start` but not `minikube delete`. When recreating the cluster, pass the proxy at start time instead:

```
minikube start --docker-env HTTP_PROXY=http://http.docker.internal:3128 \
--docker-env HTTPS_PROXY=http://http.docker.internal:3128 \
--docker-env
NO_PROXY=localhost,127.0.0.1,10.96.0.0/12,10.244.0.0/16,192.168.49.0/24,.sv
c,.cluster.local
```

Disconnecting the VPN or restarting Docker Desktop may restore direct egress and make the proxy unnecessary.